

The cassa-photometry Pipeline

From raw pixels to calibrated catalogs, with an error budget the whole way

CASSA Observatory Workshop

August 31, 2026

Where we are going

- 1 Motivation & Overview
- 2 The Data Model: SCI / ERR / DQ
- 3 Phase 1: Calibration (ISR)
- 4 Phase 2: Integration & Astrometry
- 5 Phase 3: Photometry & Zero Point
- 6 Phase 4: Diagnostics
- 7 Recap & Hands-on Preview

Why a pipeline?

A single night gives you dozens of raw frames. Each one contains, mixed together:

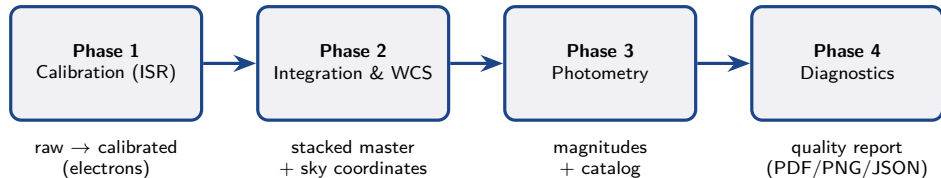
- the astronomical signal you want,
- the detector's fingerprint (bias, dark current, flat-field response),
- cosmic rays, hot/dead pixels, saturation,
- and **noise** — which never goes away, only propagates.

The pipeline turns that pile of raw FITS into **one calibrated, WCS-solved, flux-calibrated catalog** — and, crucially, tells you the **uncertainty** on every number.

The golden rule

A measurement without an error bar is not a measurement. *Every stage* of cassa-photometry carries the error forward.

The four phases



- Each phase is one `cassa-*` command and one Python sub-package.
- They share three things: a config system, logging, and the SCI/ERR/DQ FITS model.
- Each writes to its own directory: `work/phase1` → `phase2` → `phase3` → `phase4`.

One command per phase

```
cassa-calibrate -i raw/ -o work/phase1      # Phase 1: ISR
cassa-integrate work/phase1 --yes           # Phase 2: align, stack, WCS
cassa-photometry work/phase2                # Phase 3: zero point + catalog
cassa-diagnose  work/phase2 --raw raw/       # Phase 4: diagnostics

cassa-run -i raw/ -o work/                  # Phases 1->2->3 chained
cassa-verify work/phase3                    # independent sanity check
```

We will meet each of these in the hands-on session. Today: *what each one does and why*.

The conceptual spine: three planes, not one

Every science image travels as a **multi-extension FITS** file:

Plane	Meaning	Units
SCI (primary)	the science data	electrons ($\rightarrow$ Jy later)
ERR	1- σ per-pixel uncertainty	same as SCI
DQ	integer quality bitmask	—

- SCI in the *primary* HDU $\Rightarrow$ ordinary tools (DS9, `fits.getdata`, `solve-field`) still work.
- ERR and DQ ride along and are updated by *every* stage.
- This is why the final catalog can quote a real `Mag_Error`.

The DQ bitmask

A pixel can be flagged for several reasons at once — so we use **bit flags** combined with bitwise OR:

Bit value	Name	Set when...
0	GOOD	nothing wrong
1	SATURATED	pixel $\geq$ saturation ADU
2	BAD_PIXEL	hot/dead (from the bad-pixel mask)
4	COSMIC_RAY	flagged by LA Cosmic
8	NO_DATA	NaN / outside the warp footprint

Example: $DQ = 5 = 1 \mid 4 \Rightarrow$ the pixel is both saturated *and* a cosmic ray. Downstream, each catalog source inherits the OR of the flags inside its footprint.

Phase 1 — Instrument Signature Removal

Goal: remove the detector's fingerprint and *start* the error budget.

- 1 Build master **bias**, **dark**, **flat** (median-combined, each with its own uncertainty).
- 2 Build a **bad-pixel mask** from the normalized flat (outliers < 0.5 or > 1.5).
- 3 For each science frame:
 - subtract bias, subtract (scaled) dark, divide by flat;
 - reject cosmic rays (LA Cosmic / astroscrappy);
 - convert ADU $\rightarrow$ electrons via the gain;
 - assemble the DQ plane (saturation, bad pixel, CR, no-data).

Output: `calibrated_*.fits` (SCI/ERR/DQ, in electrons). Library: `ccdproc` does the arithmetic *and* propagates uncertainty automatically.

The raw uncertainty of a pixel is Poisson (photon shot noise) plus read noise:

$$\sigma_{\text{ADU}} = \frac{\sqrt{g \cdot \text{ADU} + \text{RN}^2}}{g} \quad (\text{electrons after gain: } \sigma_e = \sqrt{g \cdot \text{ADU} + \text{RN}^2})$$

where g is the gain (e^-/ADU) and RN the read noise (e^-).

Every subsequent operation propagates this in quadrature. Subtracting the master bias:

$$\sigma_{\text{out}} = \sqrt{\sigma_{\text{sci}}^2 + \sigma_{\text{bias}}^2}.$$

Dividing by the flat propagates *relative* errors:

$$\left(\frac{\sigma_{\text{out}}}{\text{sci}_{\text{out}}} \right)^2 = \left(\frac{\sigma_{\text{sci}}}{\text{sci}} \right)^2 + \left(\frac{\sigma_{\text{flat}}}{\text{flat}} \right)^2.$$

Phase 2 — align, stack, solve

Goal: combine many calibrated frames into one deep master, and give it sky coordinates.

- 1 **Group** frames by object / filter / camera / exposure.
- 2 Pick the highest-SNR frame as the **anchor**.
- 3 **Align** every other frame to the anchor (`astroalign` matches star patterns; finds shift + rotation + scale).
- 4 **Flux-scale** each frame (median ratio at matched stars), warp the data *and* its variance.
- 5 **Stack** with inverse-variance weighting + outlier rejection.
- 6 **Plate-solve** the stack with Astrometry.net (`solve-field`) to attach a WCS.

Output: `Master_*.fits` (SCI/ERR/DQ + WCS) in `work/phase2`.

If frame i has variance var_i , weight it by $w_i = 1/\text{var}_i$. The stack is

$$\text{master} = \frac{\sum_i w_i \text{sci}_i}{\sum_i w_i}, \quad \text{var}_{\text{master}} = \frac{\sum_i w_i^2 \text{var}_i}{(\sum_i w_i)^2} = \frac{1}{\sum_i w_i}.$$

- Noisy frames contribute less; clean frames dominate. This is the *optimal* linear combination.
- Stacking N equal frames shrinks the noise by $\sqrt{N}$ — the “depth boost”.
- Rejection: sigma-clipping when ≥ 5 frames, min/max otherwise, kills residual cosmic rays / satellites.

Phase 2 — the astrometry index path (HPC note)

`solve-field` needs multi-GB sky index files. The path is resolved at runtime, in order:

- 1 `phase2.astrometry_index_dir` in a config YAML, else
- 2 the `CASSA_Astrometry_INDEX` environment variable, else
- 3 `./astrometry_data`.

```
export CASSA_Astrometry_INDEX=/shared/cassa/astrometry_data
```

On the HPC we keep *one* shared read-only copy and every participant sets the env var. (There is no hard-coded path to override — the pipeline writes a fresh solver config each run.)

Phase 3 — from counts to calibrated magnitudes

Goal: put the master stack on a physical scale and catalog every source.

- 1 **Detect** stars (DA0StarFinder); measure them with aperture photometry (aperture + sky annulus), using the ERR plane.
- 2 **Cross-match** to a reference catalog; each matched star gives a zero-point estimate.
- 3 **Combine** the per-star zero points → one MAGZERO $\pm$ MAGZERR per filter.
- 4 **Flux-calibrate** the image to Janskys; **catalog** all sources (segmentation + deblending).

Output: *_fluxcal.fits (Jy), *_catalog.csv (with Flux_Error, Mag_Error, SNR, FLAGS), *_segmap.fits.

For each catalog-matched star:

$$m_{\text{inst}} = -2.5 \log_{10}(F), \quad \delta m_{\text{inst}} = \frac{2.5}{\ln 10} \frac{\delta F}{F} \approx 1.0857 \frac{\delta F}{F}.$$

$$\text{ZP}_i = m_{\text{cat}} - m_{\text{inst}}, \quad \delta \text{ZP}_i = \sqrt{\delta m_{\text{inst}}^2 + \delta m_{\text{cat}}^2}.$$

Combine with sigma-clipping + inverse-variance weighting:

$$\text{ZP} = \frac{\sum_i w_i \text{ZP}_i}{\sum_i w_i}, \quad w_i = 1/\delta \text{ZP}_i^2,$$

and report MAGZERR as the *larger* of the formal error and the scatter's standard error (conservative).
Final source magnitude:

$$\delta m = \sqrt{(1.0857 \delta F/F)^2 + \text{MAGZERR}^2}.$$

Phase 3 — the reference catalog

The zero point is calibrated *differentially*: detected stars are matched to a photometric reference catalog, queried in a filter-aware priority cascade:

APASS → Pan-STARRS DR2 → SDSS DR12

- The first catalog to return valid stars for the requested band wins.
- Each matched star contributes its catalog magnitude *and* its uncertainty.
- These are standardized into one table (RAJ2000, DEJ2000, Ref_Mag, Ref_Mag_Err).

This step reaches out to the catalog services, so the compute node needs outbound internet.

Phase 4 — did it actually work?

cassa-diagnose inspects each stage and writes a multi-page PDF + PNGs + `metrics.json`.

- **Stage 0 (raw):** PSF/FWHM, background, star detection, saturation map.
- **Stage 1 (calibrated):** SCI/ERR/DQ panels, DQ flag counts, the ERR-vs-signal (Poisson) sanity check.
- **Stage 2 (master):** SNR map, depth boost vs a single frame ($\approx \sqrt{N}$), WCS status + plate scale.
- **Stage 3 (photometry):** Mag vs Mag_Error, SNR vs Mag, number counts / limiting magnitude, star/galaxy split, the recorded ZP + MAGZERR.

This is where you *read* the error budget you have been carrying the whole way.

Recap

- The pipeline is four phases, each a `cassa-*` command.
- The SCI/ERR/DQ model is the spine: uncertainty and quality flags flow from raw pixels to final magnitudes.
- Phase 1 seeds the error; Phase 2 stacks it optimally; Phase 3 turns it into calibrated magnitudes with error bars; Phase 4 lets you check it.
- Phase 2 plate-solves against a **local** astrometry index (shared on the HPC, no network); Phase 3's catalog cross-match is a live query.

In the hands-on session

You will run all four phases on real frames in notebooks 00–05, inspecting the SCI/ERR/DQ planes and the catalog at each step.

Questions?

Next: the hands-on notebooks.